

Práctica 4

- ***Ajas Ruiz André***
- ***Colin Torres Ana Ivonne***
- ***Romero Sandoval Rodolfo Aragon***

Ejemplo A

Este ejercicio crea una variable inicializada en cero que guarda las sumas acumuladas y usa un bucle for para recorrer el arreglo. En cada paso toma el número que está en la posición *i* y se lo suma al guardado en total, después entrega el valor final acumulado.

Ejemplo B

Crea una variable en cero que sirve como contador y utiliza un bucle for que toma cada número de la lista uno por uno. La condición evalúa si el número actual es mayor a la cifra límite, si es así le suma 1 al contador y al final devuelve el conteo de los números que cumplieron con la regla.

```

import java.util.Arrays;

public class P4Arreglos {

    // ===== EJEMPLO A (resuelto) =====
    // Suma todos los numeros de un arreglo y devuelve el total.
    // Este es el recorrido clasico: la variable i es la posicion.
    static int suma(int[] numeros) {
        int total = 0;
        for (int i = 0; i < numeros.length; i++) {
            total = total + numeros[i];
        }
        return total;
    }

    // ===== EJEMPLO B (resuelto) =====
    // Cuenta cuantos numeros son mayores que el umbral.
    // Este es el for-each: se usa cuando NO necesitas la posicion, solo el valor.
    static int cuantosPasan(int[] numeros, int umbral) {
        int cuantos = 0;
        for (int valor : numeros) {
            if (valor > umbral) {
                cuantos = cuantos + 1;
            }
        }
        return cuantos;
    }
}

```

Ejercicio 1

Este ejercicio solo crea y devuelve una lista ordenada con los cinco números enteros que se establecieron. Utiliza `new int[]` para fabricar el arreglo e inicializarlo directamente.

```

// ===== EJERCICIO 1 =====
// Crea y devuelve un arreglo de int con estos cinco valores, en este orden:
// 10, 20, 30, 40, 50. Aquí no hace falta ningun for.
static int[] primerosCinco() {
    return new int[]{ 10, 20, 30, 40, 50 };
}

```

Ejercicio 2:

Calcula el promedio de una lista de números enteros y devuelve el resultado en

decimales. Utiliza un bucle for para recorrer el arreglo y sumar cada uno de los elementos, mientras double es para que el resultado se pueda dar en decimales.

```
// ===== EJERCICIO 2 =====  
// Devuelve el promedio de los numeros. Ojo con la division entera:  
// si sumas int y divides entre int, Java te devuelve int.  
// Pista: puedes apoyarte en suma(numeros), que ya esta resuelto.  
static double promedio(int[] numeros) {  
    int total = 0;  
    for (int i = 0; i < numeros.length; i++) {  
        total = total + numeros[i];  
    }  
    return (double) total / numeros.length;  
}
```

Ejercicio 3:

Se asume que el primer número es el mayor y después utiliza un bucle for para recorrer el arreglo desde la segunda posición, si encuentra un número mayor al actual, actualiza la variable max. Al terminar de recorrer toda la lista, se devuelve el número más grande que haya encontrado.

```
// ===== EJERCICIO 3 =====  
// Devuelve el numero mas grande del arreglo.  
// Pista: guarda el primero como candidato y recorre el resto comparando.  
static int maximo(int[] numeros) {  
    int max = numeros[0];  
    for (int i = 1; i < numeros.length; i++) {  
        if (numeros[i] > max) {  
            max = numeros[i];  
        }  
    }  
    return max;  
}
```

Ejercicio 4:

Utiliza un bucle for para recorrer la lista y compara cada valor buscando al valor indicado, una vez que lo encuentra devuelve inmediatamente el valor. Si no lo encuentra devuelve -1.

```
// ===== EJERCICIO 4 =====
// Devuelve la POSICION donde vive el valor buscado, o -1 si no esta.
// Si aparece dos veces, devuelve la primera.
static int posicionDe(int[] numeros, int buscado) {
    for (int i = 0; i < numeros.length; i++) {
        if (numeros[i] == buscado) {
            return i; // Devuelve la posición donde lo encontró
        }
    }
    return -1; // Si no está en el arreglo
}
```

Ejercicio 5:

Reserva memoria en un nuevo arreglo de la misma longitud y utiliza un ciclo for para recorrer el arreglo de izquierda a derecha, después asigna a la posición i del nuevo arreglo el elemento que está en la posición opuesta al arreglo original. Al final devuelve la nueva lista con los elementos invertidos.

```
// ===== EJERCICIO 5 =====
// Devuelve un arreglo NUEVO con los mismos valores al revés.
// El arreglo que recibes no se toca.
static int[] invertido(int[] numeros) {
    int[] nuevo = new int[numeros.length];
    for (int i = 0; i < numeros.length; i++) {
        nuevo[i] = numeros[numeros.length - 1 - i];
    }
    return nuevo;
}
```

Ejercicio 6:

Guarda al primer alumno de la lista, con su nombre y calificación, asumiendo que es el mejor hasta el momento, después utiliza un ciclo for para recorrer el arreglo desde la segunda posición y accede a la propiedad de calificación de cada estudiante para compararla con la del “mejor hasta el momento” si encuentra una mayor calificación, reemplaza el objeto completo en la variable mejor con los datos de este estudiante. Al terminar de revisar la lista entera, devuelve solo el nombre del mejor.

```
// ===== EJERCICIO 6 =====
// Recibe un arreglo de objetos Alumno y devuelve el nombre del que tiene
// la calificación mas alta.
static String nombreDelMasAlto(Alumno[] alumnos) {
    // Asumimos que el primer alumno tiene la calificación mas alta por ahora
    Alumno mejor = alumnos[0];

    for (int i = 1; i < alumnos.length; i++) {
        if (alumnos[i].calificacion > mejor.calificacion) {
            mejor = alumnos[i];
        }
    }
    return mejor.nombre;
}
```

Ejercicio 7:

Se indica que la variable es una matriz, se recibe el número exacto de la fila que se quiere sumar y prepara una variable encargada de guardar el total de esa suma. Utiliza un ciclo for para recorrer las columnas de la fila seleccionada y va sumando los valores. Al final devuelve el valor total sumado.

```
// ===== EJERCICIO 7 =====
// Suma los valores de UNA fila de una matriz.
// En una matriz, matriz[fila] es a su vez un arreglo: matriz[fila][columna].
static int sumaDeFila(int[][] matriz, int fila) {
    int suma = 0;

    for (int j = 0; j < matriz[fila].length; j++) {
        suma = suma + matriz[fila][j];
    }

    return suma;
}
```

Ejercicio 8:

Toma la frase completa, la divide cada que encuentra una coma y devuelve el arreglo donde mantienen sus espacios intactos, después inicializa una cadena vacía para construir la frase final. Utiliza un ciclo for para recorrer el arreglo y borrar los

espacios en blanco a los lados, después evalúa si el nombre actual es el último elemento del arreglo y si no lo es, agrega un separador.

```
// ===== EJERCICIO 8 =====  
// Recibe una línea con nombres separados por coma, posiblemente con espacios  
// de sobra, y devuelve un solo String con los nombres limpios y separados  
// por " | ". " ana , beto ,cris " -> "ana | beto | cris"  
// Pistas: linea.split(",") te devuelve un arreglo de String;  
// texto.trim() quita los espacios de las orillas.  
static String limpiaYJunta(String linea) {  
    String[] nombres = linea.split(",");  
  
    String resultado = "";  
    for (int i = 0; i < nombres.length; i++) {  
        String nombreLimpio = nombres[i].trim();  
        resultado = resultado + nombreLimpio;  
        if (i < nombres.length - 1) {  
            resultado = resultado + " | ";  
        }  
    }  
    return resultado;  
}
```